

MAC0219 - Programação Concorrente e Paralela

Mini EP4 – Profiling e otimização

Leonardo Heidi Almeida Murakami
NUSP: 11260186

26 de agosto de 2026

1 Profiling antes das otimizações

O programa original foi analisado com `gprof` e `perf`. As duas ferramentas mostraram que praticamente todo o tempo era gasto nas funções `slowsort` e `fibonacci`.

Função	gprof	perf
<code>slowsort</code>	57,55%	49,70%
<code>fibonacci</code>	42,45%	50,27%

O problema era a quantidade de chamadas recursivas. O `gprof` registrou cerca de 342 milhões de chamadas recursivas de `slowsort` e 459 milhões de `fibonacci`. Isso acontece porque o `Slowsort` é um algoritmo de ordenação muito lento e porque os mesmos números de Fibonacci eram calculados várias vezes.

2 Otimizações

Foram feitas duas mudanças:

- O `slowsort` foi trocado pelo `qsort`, que é muito mais eficiente e evita milhões de chamadas recursivas.
- Os números de Fibonacci de 0 a 35 passaram a ser calculados uma única vez e guardados em um vetor. Depois disso, cada valor pode ser consultado diretamente, sem repetir os cálculos.

Essas mudanças atacam exatamente as duas funções que apareceram como gargalos no profiling. As duas versões continuaram produzindo o mesmo resultado:

Soma dos números de Fibonacci pares: 20100614

3 Resultados depois das otimizações

A versão original levou, em média, 0,943 s no `perf stat`. A versão otimizada levou 0,527 ms. Isso representa uma aceleração de aproximadamente $1.790\times$ e uma redução de 99,94% no tempo de execução.

Métrica	Original	Otimizado
Tempo	0,943 s	0,527 ms
Ciclos	4.723.080.682	272.605
Instruções	17.030.478.110	228.259
Desvios	2.859.087.265	43.253

O `gprof` não conseguiu acumular tempo na versão otimizada, pois ela terminou antes do intervalo de amostragem de 0,01 s. O `perf record` coletou apenas 11 amostras, concentradas principalmente na inicialização e no encerramento do programa. Portanto, depois das mudanças, nenhuma função do código apareceu como um novo gargalo.